

Overload Containment Field

MIRAGE 2.0 — Binary Exploitation — Walkthrough

CATEGORY	DIFFICULTY	FILES
Binary Exploitation (Pwn)	Easy	containment_field
DEPLOYMENT	VULNERABILITY	STATUS
Offline binary — run locally	Stack buffer overflow → ret2win	Solved

Step 1 — First Contact

Running the binary gives a banner (“POWER STONE CONTAINMENT FIELD”), a warning that overload protocols are active, and a single prompt: “Enter new power level calibration:”. No menu, no visible length limit, just one line of free-form input and then the program presumably does something with it. A single unconstrained input like this, in a challenge whose objective is to “induce a catastrophic overload,” is worth treating as a buffer-overflow candidate from the very first run — that’s the shape of program this class of bug lives in.

Step 2 — Recon Before Touching Anything Else

Before writing a single line of exploit code, characterize the binary:

```
$ file containment_field
containment_field: ELF 64-bit LSB executable, x86-64, ... not stripped

$ checksec --file=containment_field
Canary: no    NX: yes    PIE: no    RELRO: partial
```

Two facts here immediately shape the plan. **Not stripped** means every function keeps its real name in the disassembly — no guessing which chunk of assembly is which. **No canary and no PIE** means there is nothing standing between an oversized write and the return address, and every address in the binary is fixed at compile time — whatever we decide to jump to, we can read its exact address straight out of the binary with no leak required. NX just rules out injecting our own shellcode onto the stack, which is fine if the binary already contains code that does what we want.

Step 3 — Reading the Disassembly

Since the binary isn’t stripped, `objdump -d -M intel containment_field` shows the function behind the prompt by name, `configure_field()`:

```
configure_field():
    sub    rsp, 0x40        ; local frame is only 64 bytes
    ...
    call   read             ; read(0, buf, 0x100)
```

The frame reserves 0x40 (64) bytes for the local buffer, but the call to `read()` is given a length of 0x100 (256) — four times the buffer's real size, with no check anywhere in between. That mismatch, visible directly in the disassembly before a single byte is ever sent, confirms the overflow.

A second thing worth checking while already reading disassembly: does anything in the binary look interesting but is never called from `main`'s control flow? Scanning the function list turns up `release_containment_field()` at 0x40123b — unreferenced by any `call` instruction anywhere. Reading it shows it prints two lines, then walks a loop XOR-ing a 36-byte embedded array with the constant 0x55 and prints the result. That's a textbook “win function”: the flag is kept obfuscated inside the binary specifically so it won't show up under `strings`, and the function that reveals it is simply never wired up to be called normally.

Step 4 — Confirming the Offset Before Building the Real Payload

With a 64-byte buffer followed by an 8-byte saved RBP and no canary, the saved return address should sit at offset 72. Rather than guessing and sending the final payload blind, confirm this cheaply first: send 72 bytes of 'A' followed by a distinctive 8-byte marker like 'BBBBBBBB' and watch the process crash under `gdb`. If RIP holds 0x4242424242424242 at the crash, the offset is confirmed — cheap insurance against silently guessing the wrong offset later.

Step 5 — Building and Running the Exploit

With the offset confirmed and the `win` function's address already known statically (no PIE, no leak needed), the payload is just three pieces: padding to fill the buffer, padding for the saved RBP (its value is never used, so it doesn't matter), and the target address.

```
from pwn import *

io = process('./containment_field')
payload = b'A' * 64      # pad to saved RBP
payload += b'B' * 8      # saved RBP (junk)
payload += p64(0x40123b) # release_containment_field()
io.sendline(payload)
io.interactive()
```

When `configure_field()` returns, it doesn't go back to `main` — it “returns” straight into `release_containment_field()`, which decodes and prints the flag.

FLAG

```
MIRAGE{w3lc0m3_t0_th3_r3t2w1n_w0rld}
```

Takeaway

Whenever a binary pairs an unchecked-length read into a stack buffer with “no canary, no PIE,” the cheapest thing to check before building any ROP chain is whether the binary already contains a function that does what you want and is simply never called — a direct return-address overwrite into that function (`ret2win`) is almost always the intended, and simplest, path.